

REAL-WORLD PROJECT • AGENTIC AI ON AWS

The AWS ChatOps Agent

Operate your AWS cloud from a chat box. Ask in plain English, let a gpt-4o agent choose the right AWS tools — and approve every change with one click.

gpt-4o **boto3** **Streamlit**

AGENT BRAIN

AWS TOOLS

LIVE UI

Prepared by Aravind Baranitharan • AI Agents for Cloud & DevOps

Why talk to your cloud in **plain English**?

The daily pain

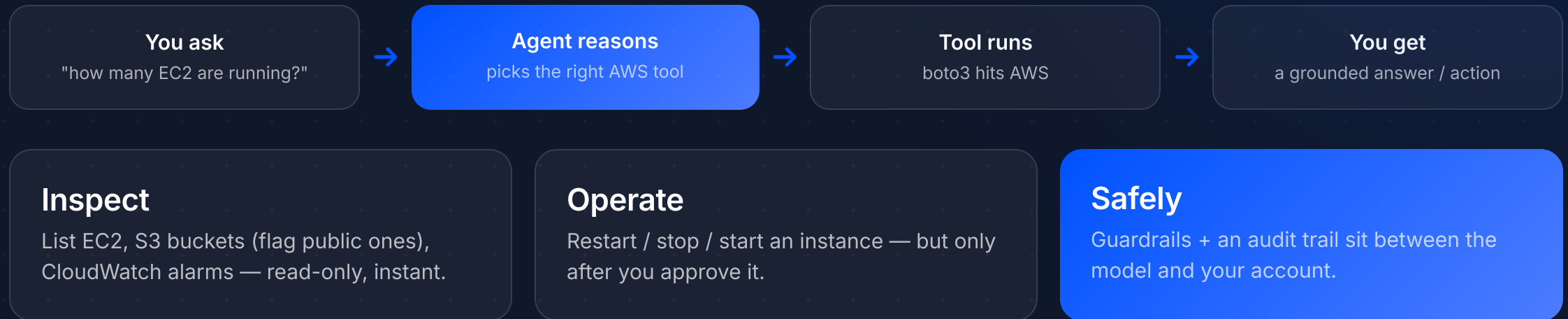
- Ops means *context-switching* : console tabs, CLI flags, dashboards.
- Answers live in people's heads — "which instance is the staging worker?"
- Manual changes are *risky* : one wrong stop-instance hits prod.

The ChatOps answer

- One chat box: *ask* and *act* in the language everyone speaks.
- The agent knows the tools — it looks things up and does the wiring.
- Every state change is *proposed first* and needs a human OK.

In one line: a conversational SRE that can read your AWS account and take *safe, approved* actions — so routine ops stop needing a console expert.

An agent that reasons, then acts



Running live against a **real AWS account**

Connected to account 836471808669 via a least-privilege IAM user. These are actual results from the demo:

- **EC2 · us-east-1** → BotxStudio (t3.small, running)
- **EC2 · ap-south-1** → markbot-production (t3.medium)
- **S3** → 13 buckets; ombak-product- as ⚠
flagged images public
- **Actions** → "restart the BotxStudio instance" → approval gate
→ done

```
You ▶ "How many running EC2 in us-east-1?"
```

```
# gpt-4o chooses a tool
```

```
AGENT ▶ list_ec2_instances(  
    region="us-east-1",  
    state="running")
```

```
# boto3 answers from your account
```

```
TOOL ◀ 1 instance: BotxStudio running
```

Same code runs offline on mock data, or live on real AWS.

The key idea: **tool calling**

The LLM never touches AWS. It only ever **asks** for a tool by name with arguments. **Your code** runs the tool with boto3 and hands back the result. That separation is what makes an agent both powerful and safe.

- A tool = a **name** + a **description** + a typed **schema** .
- The model reads the description to decide *when* to call it.
- You stay in control of *what actually executes* .

The round-trip

model: "call restart_ec2_instance(i-03b...)"

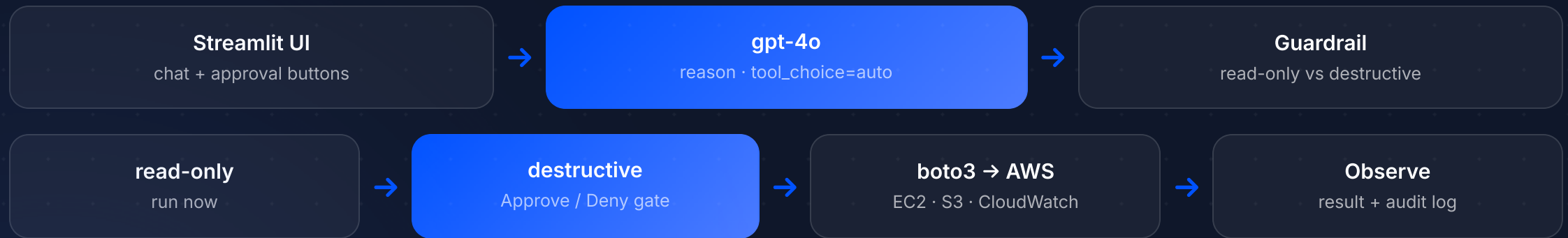
your code executes it (boto3)

result returned to the model

model explains what happened

No tool = no action. The model can only do what you gave it — and destructive tools wait for you.

End-to-end data flow



Loop, bounded: the model may chain tools (look up an id, then act), but the loop is capped at 8 steps and no state-changing call runs without an explicit approval.

Perceive → reason → **act** → repeat

Each turn, the whole conversation (system + history + tool results) is re-sent to gpt-4o. If it returns `tool_calls`, we run them and loop; if it returns text, we're done. That single loop drives both the live model and the offline fallback.

- **temperature = 0** → deterministic, demo-safe.
- Tool errors come back as *observations*, not crashes.
- A hard **step cap** means the loop can never run away.

```
resp = client.chat.completions.create(
    model="gpt-4o", messages=messages,
    tools=TOOLS, tool_choice="auto",
    temperature=0)

msg = resp.choices[0].message
# the model asked for tools?
for tc in msg.tool_calls:
    out = execute_tool(tc.name, tc.args)
    messages.append(tool_result(tc.id, out))
```

agent.py · `call_model()` – the core round-trip

The agent proposes — a human approves

Every tool carries a **risk label**. Read-only tools run automatically; anything that changes state is **held** and surfaced as an Approve / Deny button in the UI. Nothing mutates your account silently.

- **Classification** — read-only vs destructive, per tool.
- **Approval gate** — the loop pauses for a human click.
- **Audit log** — every action, decision & timestamp recorded.

```
RISK = {  
    "list_ec2_instances": "read_only",  
    "get_cloudwatch_alarms": "read_only",  
    "restart_ec2_instance": "destructive",  
    "stop_ec2_instance": "destructive",  
}  
  
# destructive → pause the loop for approval  
if RISK[name] == "destructive":  
    st.session_state.pending = call
```

aws_tools.py + app.py — one map drives the whole gate

Services, and two ways to **run it**

EC2

List instances by region/state; restart · stop · start (gated).

S3

List buckets and flag any that are publicly accessible.

CloudWatch

List alarms, optionally filtered by state (ALARM / OK).



Mock mode

USE_REAL_AWS=false — deterministic fake inventory. Zero risk, no account needed. Perfect for teaching and on-stage demos.



Real mode

USE_REAL_AWS=true — boto3 against your account via the standard AWS credential chain. Same code, real resources.

Six files, **clear responsibilities**

FILE	RESPONSIBILITY
aws_tools.py	The AWS tools (mock + real boto3) + OpenAI tool schemas + the RISK map.
agent.py	The gpt-4o tool-calling brain (<code>call_model</code>) + an offline fallback.
app.py	The Streamlit UI: chat, the loop driver, the Approve/Deny gate, audit log.
test_connection.py	Read-only pre-flight: verifies creds + each IAM permission.
iam-policy.json	Least-privilege IAM policy — exactly the actions the app uses.
env.sample · requirements.txt	Config template (rename to <code>.env</code>) + dependencies.

A tool = function + **schema**

Each tool is a plain Python function that works in **mock** or **real** mode, plus an OpenAI schema the model reads. In real mode it's a one-line boto3 call.

- TOOLS → the schemas passed to gpt-4o.
- REGISTRY → name → the Python function.
- RISK → name → read-only / destructive.

```
def restart_ec2_instance(instance_id, region=None):
    region = region or DEFAULT_REGION
    if _use_real():
        boto3.client("ec2", region_name=region)\
            .reboot_instances(InstanceIds=[instance_id])
        return f"Reboot requested for {instance_id}"
    ... # mock path for offline demos

REGISTRY = {"restart_ec2_instance": restart_ec2_instance, ..
```

One function, two modes – mock for demos, boto3 for real.

The brain, UI-agnostic

`call_model()` runs one turn and returns a normalized result the UI can drive: the final text, or the list of tool calls the model wants. It works with a real key (gpt-4o) and has a small offline fallback so it demos with no key.

- Live: OpenAI SDK, `tools=TOOLS` , `tool_choice="auto"` .
- `execute_tool()` runs a chosen tool safely.
- Config (key, model, region) loaded from `.env` .

```
def call_model(messages):  
    # → (content, calls[{id,name,args}], assistant)  
    if has_key():  
        return _live(messages)    # real gpt-4o  
    return _offline(messages)    # keyword fallback  
  
def execute_tool(name, args):  
    return REGISTRY[name](**args)    # run the boto3 tool
```

Same interface, live or offline – the UI never changes.

Streamlit UI & the **approval gate**

`drive()` runs model turns, auto-executing read-only tools, until a final answer or a destructive action needs approval. Then it pauses and the UI renders the buttons; `resolve()` finishes the action on your click.

- Chat history + tool traces rendered as messages.
- **Approve** → run it & log; **Deny** → skip, log the block.
- Sidebar shows mode, brain, and the live audit log.

```
for c in calls:
    if RISK[c.name] == "destructive":
        st.session_state.pending = c    # pause
    else:
        out = execute_tool(c.name, c.args)    # run now

# on the human's click:
if approve: execute_tool(c.name, c.args); audit("approved")
if deny:    audit("denied")    # nothing runs
```

The whole safety story, in a dozen lines.

Least privilege, **by design**

Scoped IAM

1

A dedicated user with only the actions the app calls.

No secrets in code

2

Keys live in `.env` (`chmod 600`, `git-ignored`).

Human gate

3

No restart/stop/start without an explicit approval.

Audit trail

Every action + decision, timestamped.

Read-only first: a pre-flight (`test_connection.py`) verifies each permission before you demo. Start with inspect-only tools, add actions deliberately.

A safe, conversational **cloud operator**

- ✓ Plain-English ops over EC2, S3 & CloudWatch.
- ✓ gpt-4o + tool calling + boto3 — the model reasons, your code acts.
- ✓ Guardrails: read-only runs free, changes need approval.
- ✓ Runs live on real AWS or fully offline on mock data.
- ✓ `streamlit run app.py` — that's the whole demo.

WHERE IT GOES NEXT

From ChatOps to autonomy

Add a **FinOps** tool (idle spend), then wire alarms → the agent to build **auto-remediation** — detect, propose, approve, fix, verify.

AGENTIC OPS → SELF-HEALING